

A Deep Reinforcement Learning Approach for Synchronization Between Two Memristor Chaotic Systems and Application for Image Encryption

Shitao Jin, Jie Chen[✉], Jie Wu[✉], Xiaofeng Wang, Xiaoli Luan[✉], *Member, IEEE*,
Junjie Fu[✉], *Senior Member, IEEE*, and Guanghui Wen[✉], *Senior Member, IEEE*

Abstract—This study proposes a novel synchronization framework for memristive chaotic systems (MCSs) through an enhanced deep reinforcement learning (DRL) approach, featuring an improved proximal policy optimization (PPO) algorithm. Distinguished from traditional linear/nonlinear control paradigms that necessitate precise mathematical modeling, our DRL-based methodology operates without prior knowledge of system dynamics or analytical model requirements. The developed data-driven control strategy demonstrates significant advantages by reducing the required control forces from four to three dimensions, thereby substantially decreasing control complexity and operational costs compared to conventional item-by-item control methods. Through systematic optimization of the reward function architecture in classical PPO algorithms, we achieve accelerated synchronization convergence rates for MCSs, in which an optimal exponential parameter is obtained accordingly. Finally, the practical efficacy of our DRL-driven synchronization framework is successfully validated in image encryption applications. Comprehensive numerical simulations and comparative analyses demonstrate that the proposed methodology not only maintains robust performance under Gaussian noise perturbations but also achieves synchronization efficiency improvements.

Index Terms—Memristor chaotic circuits, deep reinforcement learning, synchronization, image encryption.

I. INTRODUCTION

CHAOTIC systems, characterized as deterministic nonlinear dynamical systems with extreme sensitivity to initial conditions, generate seemingly stochastic behaviors despite their inherent mathematical predictability. The foundational understanding of chaos theory emerged from Edward Lorenz's seminal 1960s meteorological simulations [1], which revealed that infinitesimal perturbations in initial states could yield exponentially divergent outcomes—a paradigm now ubiquitously referenced as the “butterfly effect”. Subsequent investigations have extensively documented chaotic phenomena across multidisciplinary domains, including physics [2], chemistry [3], biology [4], and financial market analyses [5], with comprehensive theoretical frameworks established in authoritative surveys [6].

One prominent application lies in cryptography, where the inherent unpredictability of chaotic attractors is exploited to design permutation-substitution architectures with enhanced avalanche effects, thereby improving encryption security and diffusion properties [7]. In the domain of secure communications, chaos-based spread spectrum techniques have been shown to achieve superior bit error rate performance compared to conventional modulation schemes, offering greater robustness against noise and interference [8]. Beyond information security, chaotic dynamics have also found meaningful applications in biological modeling, particularly in the study of complex physiological rhythms such as cardiac arrhythmias and neural synchronization patterns [9]. These diverse applications underscore the broad utility of chaos theory in addressing real-world challenges across multiple domains.

As a newly emerging concept within the realm of nonlinear dynamics, memristive chaotic systems (MCSs) have arisen as a revolutionary framework in contemporary chaos research. Chua first postulated the memristor as the fourth fundamental circuit element in 1971 [10], and its physical realization was achieved by Hewlett-Packard (HP) Labs in 2008 [11]. Since then, numerous studies have demonstrated that MCSs exhibit heightened dynamical complexity and extreme sensitivity to initial conditions [12], [13], prompting extensive exploration into their modeling, control, and application [14], [15], [16], [17], [18]. Significant progress has been made in

Received 20 April 2025; revised 13 June 2025 and 2 July 2025; accepted 16 July 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62103166, Grant 62172191, Grant 62173085, Grant 62325304, Grant U22B2046, and Grant U24A20279; in part by Jiangsu Science Fund for Distinguished Young Scholars under Grant BK20240072; in part by Jiangsu Provincial Scientific Research Center of Applied Mathematics under Grant BK20233002; in part by the Shuangchuang Doctoral Talent Program of Jiangsu Province under Grant JSSCBS20210840; and in part by the Southeast University Interdisciplinary Research Program for Young Scholars. This article was recommended by Associate Editor Y. Tang. (*Corresponding author: Jie Wu.*)

Shitao Jin is with the School of Artificial Intelligence and Computer Science, Jiangnan University, Wuxi 214122, China (e-mail: 6233114014@stu.jiangnan.edu.cn).

Jie Chen is with the College of Electronic Engineering, National University of Defense Technology, Hefei 230027, China (e-mail: jchen202209@163.com).

Jie Wu is with the School of Artificial Intelligence and Computer Science, Jiangnan University, Wuxi 214122, China, and also with the Peng Cheng Laboratory, Shenzhen 518000, China (e-mail: jiewu20@jiangnan.edu.cn).

Xiaofeng Wang is with the Peng Cheng Laboratory, Shenzhen 518000, China (e-mail: wangxf@pcl.ac.cn).

Xiaoli Luan is with the Key Laboratory of Advanced Process Control for Light Industry, Ministry of Education, School of Internet of Things Engineering, Jiangnan University, Wuxi 214122, China (e-mail: xlluan@jiangnan.edu.cn).

Junjie Fu is with the Department of Systems Science, School of Mathematics, Southeast University, Nanjing 211189, China (e-mail: fujunjie@seu.edu.cn).

Guanghui Wen is with the School of Automation, Southeast University, Nanjing 210096, China (e-mail: ghwen@seu.edu.cn).

Digital Object Identifier 10.1109/TCSI.2025.3592187

synchronizing MCSs using various control strategies. Adaptive controllers have successfully synchronized memristor-based Chua's circuits [19], while fuzzy modeling approaches have been applied to analyze and synchronize such systems under uncertainty [20]. Advanced sliding mode control techniques have enabled fixed-time synchronization in fractional-order memristive neural networks [21]. Recent efforts have extended to adaptive synchronization of delayed systems [22] and intermittent stabilization via a Takagi and Sugeno (T-S) fuzzy model [23], demonstrating improved robustness and flexibility.

The inherent unpredictability and extreme sensitivity of MCSs have made them particularly valuable for image encryption applications. Initial developments focused on MCS-based decryption algorithms [24] and sophisticated chaotic network encryption schemes [25], with finite-time synchronization further enhancing performance [26]. Subsequent architectural innovations introduced multi-memristor circuits [27] and specialized color image encryption systems [28], while hybrid approaches combining MCSs with traditional cryptography achieved enhanced security [29], [30]. Most recently, advanced control frameworks using sliding-mode synchronization [31] and novel 3D memristive cubic maps with dual memristors [32] have demonstrated significant improvements in both security and efficiency for image protection.

However, traditional methods employed for achieving synchronization in MCSs heavily rely on precise knowledge of system equations — a condition that is rarely met in practical applications due to factors such as approximate modeling, inaccurate observations, data transmission loss, and external noise disturbances. To address this limitation, deep reinforcement learning (DRL) has emerged as a promising alternative by enabling synchronization without the need for explicit mathematical models or prior analytical knowledge. This model-free approach significantly relaxes the constraints imposed by classical control strategies and enhances adaptability in uncertain or dynamic environments. Notably, Bucci et al. applied the deep deterministic policy gradient (DDPG) algorithm — a representative DRL method — to stabilize a one-dimensional chaotic Kuramoto-Sivashinsky system [33], demonstrating the feasibility of model-free control in chaotic settings. Similarly, Chen et al. developed a DRL-based approach to achieve synchronization between two identical chaotic systems [34], further validating the potential of these algorithms in complex dynamical control tasks. Collectively, the studies cited in [33], [34], [35], [36], and [37] have laid a solid foundation for applying DRL techniques to chaotic synchronization.

Despite these achievements [33], [34], [35], [36], [37], there are still many urgent issues that have not been effectively resolved. Firstly, no research has focused on the synchronization of MCSs based on DRL as well as their application in image encryption. Secondly, the existed studies have used item-by-item control and have not focused on reducing the cost and complexity of control inputs. Thirdly, the traditional reward functions have been directly used without effective improvement, resulting in a slow synchronization convergence speed of systems. These gaps will be bridged in this investigation. Therefore, this paper endeavors to tackle the

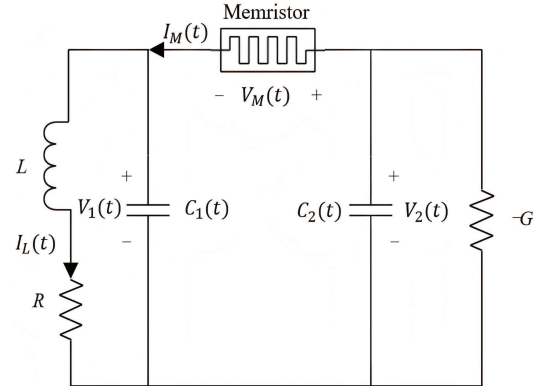


Fig. 1. The diagram of memristor chaotic circuit [31].

synchronization challenge of MCSs through DRL methodologies. Additionally, an image encryption algorithm grounded in MCSs is put forth. The overarching goal is to capitalize on the distinctive features of MCSs—such as their pronounced sensitivity to initial conditions and intricate dynamics—to fortify the security and resilience of the encryption algorithm. The key contributions of this study are delineated as follows:

- (1) A novel DRL approach, anchored in the proximal policy optimization (PPO) algorithm, is proposed to resolve the synchronization problem of MCSs, which is distinguished from the traditional methods requiring the prior knowledge of system dynamics or analytical model.
- (2) Departing from the conventional practice of item-by-item control (involving four control terms) during MCSs synchronization, this investigation streamlines the control architecture by reducing the number of control forces to three. This optimization effectively curtails the control cost.
- (3) This paper enhances the classical PPO algorithm for agent training. Specifically, the reward function is refined through the introduction of an additional term. This modification enables MCSs to achieve synchronization with a rapid convergence rate and endows them with robustness against Gaussian noise interference.
- (4) The improved DRL method for achieving synchronization of MCSs is successfully applied into image encryption, thereby broadening the application domain of DRL algorithms.

The following structure of this article is organized as follows. Section II introduces the MCSs and the enhanced PPO algorithm. Sections III and IV present the experimental results and the application in image encryption. Finally, the conclusions and future directions are given in Section V.

II. THE MODEL OF MCSs AND DESIGNING OF DRL ALGORITHM

A. Model Description of MCSs

For simplicity and without loss of generality, this article considers the following memristor chaotic circuit [31] as illustrated in Fig. 1. Therefore, based on Chua's circuit and

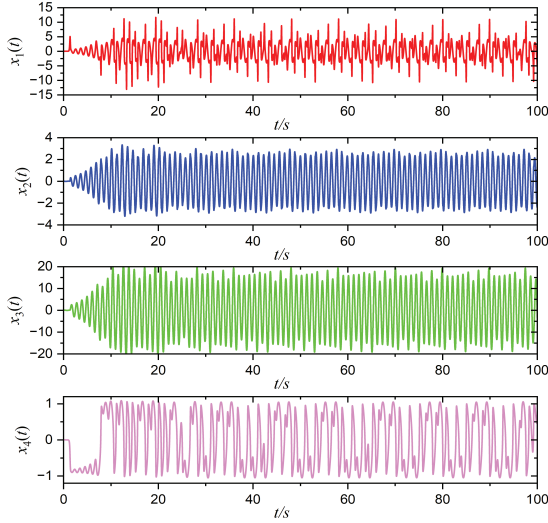


Fig. 2. The trajectories of state variables $x_i(t)$, $i = 1, 2, 3, 4$, of system (2).

Kirchhoff's current law [38], the dynamical model of the studied MCS is governed by

$$\begin{cases} \dot{V}_1(t) = \frac{1}{C_1} [\eta z^2(t)(V_2(t) - V_1(t)) + G V_1(t)], \\ \dot{V}_2(t) = \frac{1}{C_2} [\eta z^2(t)(V_1(t) - V_2(t)) - I_L(t)], \\ \dot{I}_L(t) = \frac{1}{L}(V_2(t) - R I_L(t)), \\ \dot{z}(t) = (V_2(t) - V_1(t)) - \zeta z(t), \end{cases} \quad (1)$$

where $V_1(t)(V_2(t))$ denotes the instantaneous voltage across capacitor $C_1(C_2)$, $-G$ signifies a negative conductance, L is an inductor, R is a common resistor, and $I_L(t)$ indicates the current flowing through both the resistor R and the inductor L .

Let $x_1(t) = V_1(t)$, $x_2(t) = V_2(t)$, $x_3(t) = I_L(t)$, $x_4(t) = z(t)$, $a = 1/C_1$, $b = 1/C_2$, $c = 1/L$, $d = G$, $h = R$, $\eta = 2$, $\zeta = 0.01$, then the system (1) can be rewritten as

$$\begin{cases} \dot{x}_1(t) = a [2 x_4^2(t)(x_2(t) - x_1(t)) + d x_1(t)], \\ \dot{x}_2(t) = b [2 x_4^2(t)(x_1(t) - x_2(t)) - x_3(t)], \\ \dot{x}_3(t) = c(x_2(t) - h x_3(t)), \\ \dot{x}_4(t) = x_2(t) - x_1(t) - 0.01 x_4(t). \end{cases} \quad (2)$$

To ensure the MCS (2) is programmable, we consider its discrete-time form as follows:

$$\begin{cases} x_1(t+1) = x_1(t) + \dot{x}_1(t)dt, \\ x_2(t+1) = x_2(t) + \dot{x}_2(t)dt, \\ x_3(t+1) = x_3(t) + \dot{x}_3(t)dt, \\ x_4(t+1) = x_4(t) + \dot{x}_4(t)dt. \end{cases} \quad (3)$$

We set the parameters of system (2) as $a = 30$, $b = 1$, $c = 36$, $d = 0.5$, $h = 0.003$ and the initial state $[x_1, x_2, x_3, x_4]^T = [0, 0.01, 0.01, 0]^T$. Figure 2 shows the trajectories of state variables $x_i(t)$, $i = 1, 2, 3, 4$, of system (2), and the 3-D phase diagrams are depicted in Fig. 3.

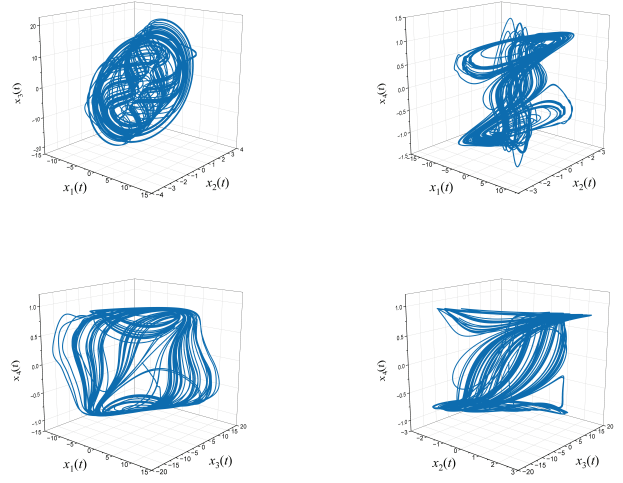


Fig. 3. The chaotic phase diagrams of system (2).

Since this investigation considers the synchronization of MCSs, we set Eq. (2) as the drive system and the corresponding controlled response system is described as

$$\begin{cases} \dot{y}_1(t) = a [2 y_4^2(t)(y_2(t) - y_1(t)) + d y_1(t)] + a_1(t), \\ \dot{y}_2(t) = b [2 y_4^2(t)(y_1(t) - y_2(t)) - y_3(t)] + a_2(t), \\ \dot{y}_3(t) = c(y_2(t) - h y_3(t)) + a_3(t), \\ \dot{y}_4(t) = y_2(t) - y_1(t) - 0.01 y_4(t) + a_4(t), \end{cases} \quad (4)$$

where $a_i(t)$, $i = 1, 2, 3, 4$, are the control forces to be trained. For measuring synchronization, we define the errors $e_i(t) = x_i(t) - y_i(t)$, $i = 1, 2, 3, 4$, and we can get the following error dynamic system

$$\begin{cases} \dot{e}_1(t) = a [2 y_4^2(t)(y_2(t) - y_1(t)) - 2 x_4^2(t)(x_2(t) - x_1(t)) + d e_1(t)] + a_1(t), \\ \dot{e}_2(t) = b [2 y_4^2(t)(y_1(t) - y_2(t)) - 2 x_4^2(t)(x_1(t) - x_2(t)) - e_3(t)] + a_2(t), \\ \dot{e}_3(t) = c(e_2(t) - h e_3(t)) + a_3(t), \\ \dot{e}_4(t) = e_2(t) - e_1(t) - 0.01 e_4(t) + a_4(t). \end{cases} \quad (5)$$

Remark 1: The synchronization problem of drive-response MCSs (2) and (4) is transformed into the stability analysis of error dynamic system (3).

Definition 1: The synchronization between the drive-response MCSs (2) and (4) is achieved, if

$$\lim_{t \rightarrow \infty} |x_i(t) - y_i(t)| \triangleq \lim_{t \rightarrow \infty} |e_i(t)| \rightarrow 0, \quad i = 1, 2, 3, 4.$$

Remark 2: For the response system (4) and the error system (3), we also consider their discrete-time forms in programming.

B. The Enhanced PPO Algorithm

In the realm of artificial intelligence (AI), one of the central objectives is to engineer systems that can render optimal decisions within intricate and ever-evolving environments. Conventional algorithms frequently encounter challenges when dealing with high-dimensional data and necessitate explicit programming rules to operate efficiently.

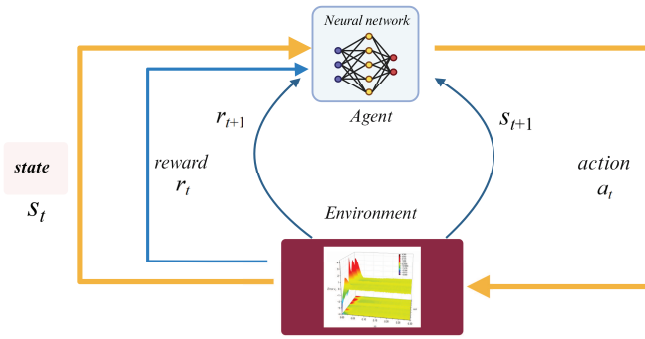


Fig. 4. The framework of DRL.

This is precisely where reinforcement learning (RL) demonstrates its exceptional utility. Through continuous interaction with their environment and receiving feedback in the form of rewards or penalties, RL empowers agents to learn and execute optimal behaviors.

At the core of deep learning (DL) lie deep neural networks. These networks are capable of learning high-level abstract representations of data via multiple layers of nonlinear transformations. This enables DL models to capture complex patterns and features inherent in the data. The integration of neural networks with RL substantially bolsters RL's ability to address high-dimensional problems. Referred to as DRL, this convergence capitalizes on the advantages of both methodologies.

In DRL, the primary goal of an agent is to continually refine its decision-making abilities through learning from experience. The agent aims to execute a sequence of actions that ultimately maximizes cumulative rewards. The DRL framework, illustrated in Fig. 4, outlines the fundamental interaction loop between an agent and its environment. This interaction occurs over a series of discrete time steps, denoted as t . At each time step t , the agent first observes the current state of the environment s_t and receives r_t as a reward, then selects an action a_t based on its policy. The chosen action influences the environment, leading to a transition to a new state s_{t+1} and the generation of a reward signal r_{t+1} , which provides feedback on the quality of the action taken. In this framework, the environment represents the error dynamic system (3), while the agent is represented by a neural network trained using the PPO algorithm.

Remark 3: Given its powerful functionality, robust performance, and straightforward implementation, we choose the PPO algorithm to control the error dynamic system (3) in this investigation.

As the PPO algorithm builds upon policy gradient methods as its foundation, we begin with an introduction of these fundamental RL techniques. Policy gradient methods constitute a class of model-free RL approaches designed to enable autonomous agents to learn optimal action selection strategies that maximize cumulative rewards. The core mechanism involves policy gradient estimation, where agents systematically compute gradients of expected returns with respect to policy parameters, subsequently integrating these gradients

into a stochastic gradient ascent optimization framework. The most commonly used form of the gradient estimator $\hat{g}$ is structured as follows [39]:

$$\hat{g} = \hat{\mathbb{E}}_{s_t \sim \rho(\cdot|\pi)} [\nabla_{\theta} \log \pi(a_t|s_t) \hat{A}_t], \quad (6)$$

where the expectation $\hat{\mathbb{E}}[\cdot]$ denotes the empirical average over a batch of finite samples, π represents a stochastic policy that defines the probability distribution over actions given states, $\rho(\cdot|\pi)$ denotes the distribution of states encountered when an agent follows policy π , ∇_{θ} represents the policy gradient, θ denotes the current policy parameters, and $\hat{A}_t$ represents the advantage function, measuring how much better the chosen action is compared to the average action in a given state.

PPO, as an advanced policy gradient method, has become a predominant solution for policy optimization challenges in continuous action spaces, particularly effective in high-dimensional continuous control tasks. The algorithm manifests in two distinct algorithmic implementations: the Lagrangian penalty-based variant (PPO-Penalty) and the clipped probability ratio method (PPO-Clip). The rationale for selecting PPO-Clip principally stems from its theoretically-grounded advantages: (1) enhanced training stability through bounded policy updates, (2) implementation simplicity eliminating adaptive penalty coefficient tuning, (3) computational efficiency suitable for large-scale learning paradigms, (4) generalized applicability across diverse environment configurations, and (5) robust community-driven development ecosystem facilitating reproducible implementations. These combined merits motivate our adoption of PPO-Clip as the core optimization framework in the present study. The objective function of the PPO-Clip algorithm is presented as following [39]:

$$L(s, a, \theta_k, \theta) = \mathbb{E}_{s, a \sim \pi_{\theta_k}} \left[\min \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_k}(a|s)} A^{\pi_{\theta_k}}(s, a), \right. \right. \\ \left. \left. \text{clip} \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_k}(a|s)}, 1 - \epsilon, 1 + \epsilon \right) A^{\pi_{\theta_k}}(s, a) \right) \right], \quad (7)$$

where L stands for the objective function, a is the action, θ_k represents the parameters before the update, $A^{\pi_{\theta_k}}$ is the advantage function, the clip function limits the probability ratio between the new and old policies, and ϵ is a hyperparameter. According to [39], the PPO algorithm clips the policy ratio $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ within $[1 - \epsilon, 1 + \epsilon]$ to prevent excessively large policy updates that could destabilize training. The symmetric range ensures balanced constraints for both increased and decreased action probabilities. The default $\epsilon = 0.2$ was empirically found to optimally balance learning speed and stability across various environments, though values between 0.1-0.3 are commonly used depending on task sensitivity. This clipping mechanism provides a simpler alternative to the constrained trust region policy optimization while maintaining similar performance guarantees.

To evaluate the performance of an action in comparison to the average performance, it is necessary to calculate the advantage function. For this task, we select the generalized advantage estimation (GAE) [40]:

$$A^{\pi_{\theta_k}}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s), \quad (8)$$

where Q represents the action-value function and V stands for the state-value function. When $A^{\pi_{\theta_k}} > 0$, it implies that the current policy is more effective than the previous one, thereby increasing the probability of generating the trajectory. However, this increase is restricted such that the parameters θ and θ_k are maintained within the range of $1 + \epsilon$. On the contrary, when $A^{\pi_{\theta_k}} < 0$, it decreases the probability of generating trajectories while ensuring that the parameters θ and θ_k do not fall below the range of $1 - \epsilon$.

In this study, we utilize the DRL method to train an agent which serves as the controller, substituting traditional control techniques. From the DRL viewpoint, this problem can be formulated as guiding an agent to get a suitable action policy, enabling the system's output vector to attain a objective state. The formulation of the action policy generated by the agent is presented as follows:

$$a_t = [a_1, a_2, a_3, a_4], -f_{\max} \leq a_i \leq f_{\max}, i = 1, 2, 3, 4, \quad (9)$$

in which we set $a_3 = 0$ for decreasing control complexity and operational costs, $f_{\max}$ is the maximum value of the action vector. The initial values for the environment are obtained from a random distribution.

Reducing one control dimension can effectively decrease the computational load per iteration, resulting in faster simulation time. More importantly, reducing the dimensionality also decreases the output dimension of the deep neural networks, thereby improving both the training and inference speed of the model. In real-time systems, this reduction enhances response speed by reducing latency.

Remark 4: The traditional linear and nonlinear methods mainly adopted item-by-item control, that is, $a_i \neq 0, i = 1, 2, 3, 4$, before synchronization is achieved. In this study, we adopt a DRL-based method to break through this bottleneck and minimize the number of control terms as much as possible so as to streamline the control process and cut down on control cost. Specifically, we have attempted to fix any of the control parameters a_1, a_2, a_3, a_4 at 0. However, experimental results show that only when a_3 is fixed at 0 can we achieve the desired outcomes.

The termination criterion for the environment training is as follows: either reaching the predefined number of time steps or when the reward becomes exceedingly small.

At the commencement of the agent's training, closely monitoring its performance throughout the training process is of utmost significance. Consequently, the proper setting of observation values holds particular importance. The observation value O_t at the present time step is defined in the following manner:

$$O_t = [e_1(t), e_2(t), e_3(t), e_4(t), \dot{e}_1(t), \dot{e}_2(t), \dot{e}_3(t), \dot{e}_4(t)]^T. \quad (10)$$

Merely taking into account the current position is insufficient to comprehensively and thoroughly evaluate whether the current state corresponds to the desired target state. Thus, our observations incorporate not only the current errors but also their derivatives.

The design of the reward function plays a crucial role during the agent's training process. A well-designed reward function

enables efficient training of the model. In our study, the reward function is formulated as:

$$\text{reward} = \sum_{t=1}^T r_t, \quad (11)$$

where $r_t = -(|e_1(t)| + |e_2(t)| + |e_3(t)| + |e_4(t)|) - (|e_1(t)| + |e_2(t)| + |e_3(t)| + |e_4(t)|)^\alpha$ quantifies the cumulative value of positional errors. The exponential parameter $\alpha \in (0, 1)$. Consequently, as the reward r_t approaches zero, it signifies a closer convergence towards the stable point. Since our principal objective is to guide the error dynamic system towards a stable state, we expect to observe a declining trend in the error values. However, the agent is designed to find sequences of actions that maximize rewards. This goal conflict with our aim to minimize system errors. To address this issue, we use the negative of the positional errors. By doing so, we ensure that the system error is minimized while still keeping the reward values high.

Remark 5: This reward formulation offers several advantages for PPO-based training. First, by introducing the non-linear term with exponent α , it emphasizes sparsity in the error space, particularly when $0 < \alpha < 1$. This encourages the agent to prioritize states where errors can be significantly reduced, similar to the behavior observed in ℓ^p -norm minimization with $0 < p < 1$. Second, the structure helps balance exploration and exploitation by reducing the impact of large errors while amplifying smaller ones, promoting stable and efficient learning. Lastly, the tunable parameter α provides flexibility, allowing the reward sensitivity to be adapted for different tasks and environments, thereby enhancing the overall robustness and performance of the learned policy.

In sum, Fig. 5 shows the whole training process of our method based on PPO algorithm. Additionally, as illustrated in Algorithm 1, we outline a procedure for implementing the PPO algorithm in the error dynamic system (3). This process involves using m_t to approximate the mean of the gradients, which is calculated as $m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_\phi L(\phi_t)$, where β_1 is the decay rate for the first moment estimates. Additionally, v_t is employed to estimate the uncentered variance of the gradients, computed by $v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_\phi L(\phi_t))^2$, with β_2 being the decay rate for the second moment estimates. Bias correction is applied through $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$ and $\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$, ensuring more accurate updates. The parameters are then updated using these bias-corrected estimates to achieve more stable convergence, following $\phi_{t+1} = \phi_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$. The resultant experimental findings are discussed in detail in the following section.

III. EXPERIMENTAL RESULTS

This section provides the experimental details on a computer equipped with a ray tracing textel (RTX) 4090D. All programs are written in Python. The training process covers a duration of 500,000 time steps. For the differential equation resolution, each time step is 0.001 unit. The control force is constrained to the interval $[-200, 200]$. The initialization of the system is carried out with variable values sampled from the range $[0, 5]$. The learning rate is 0.0003, and the policy is implemented

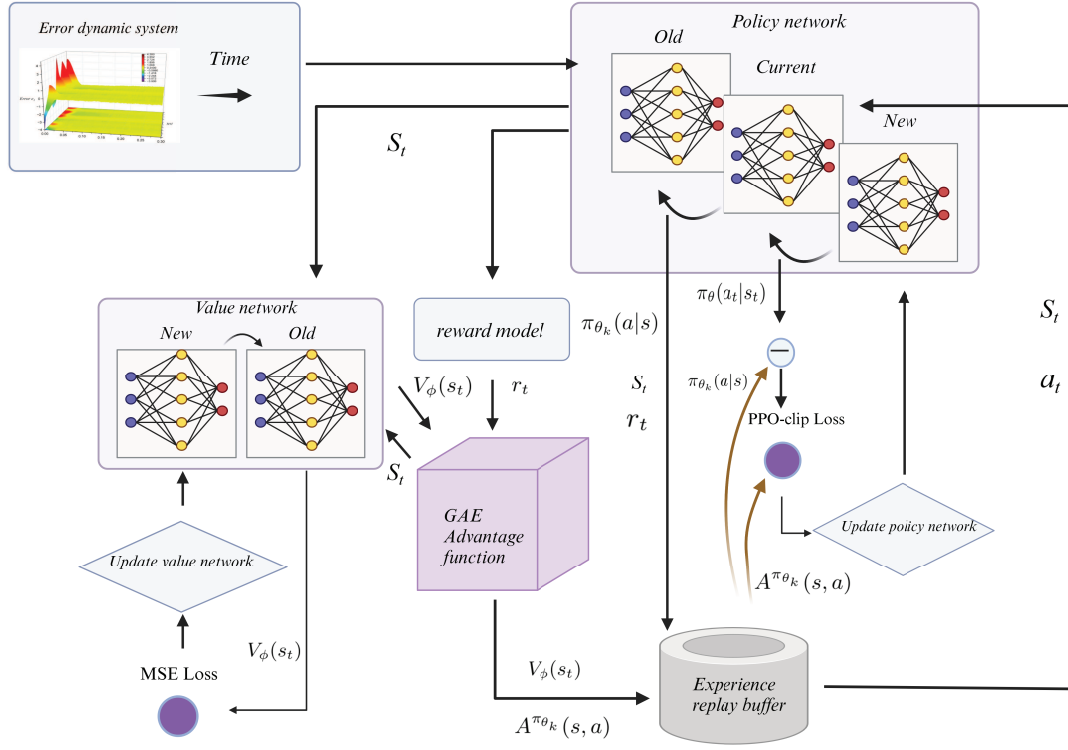


Fig. 5. The training process of our method based on PPO algorithm.

using a multi-layer perceptron. The exponential parameter α is set as $1/2$.

Firstly, regarding our contribution (2), we conducted test experiments with the number of control forces set to 3 and 4. The test process covered a duration of 5,000 time steps. Initially, we simulated the scenario where the control forces were randomly generated (i.e., not produced by our model, but assigned random values). In this case, when the number of control forces was 3, the execution time was 0.0956 s; when it was increased to 4, the execution time was 0.1053 s. When the control forces were instead generated through our model, the computational cost naturally increased due to the inference process. Specifically, for 3 control forces, the execution time was 2.0181 s; for 4 control forces, it was 2.4707 s. It is evident that reducing one control dimension significantly decreases the computational load per iteration, resulting in faster simulation times. By comparing the performance with and without passing through our model, we aim to demonstrate that the reduction in the number of control forces brings greater improvement within the DRL framework.

In machine learning, the root-mean-square error (RMSE) and mean absolute error (MAE) are usually utilized to evaluate the difference between the predicted outputs and the true values, which are given as following:

$$\text{RMSE} = \sqrt{\frac{\sum_{i=1}^N (\hat{p}_i - p_i)^2}{N}}, \quad (12)$$

$$\text{MAE} = \frac{\sum_{i=1}^N |\hat{p}_i - p_i|}{N}, \quad (13)$$

TABLE I
MAE AND RMSE WITH OUR PROPOSED REWARD (11)

	e_1	e_2	e_3	e_4	Average
MAE	0.018716	0.012133	0.009118	0.291184	0.082788
RMSE	0.027032	0.015278	0.011555	0.120661	0.053696
MAE-noise	0.017284	0.012047	0.009527	0.293319	0.083044
RMSE-noise	0.023681	0.015168	0.012025	0.121103	0.053319

where N is the total number of data points, $\hat{p}$ is the position of error dynamic system, and p represents the objective value. In our study, we use the MAE and RMSE as the primary performance comparison metrics of e_1 , e_2 , e_3 , e_4 and their averages.

In our initial analysis, we evaluate the convergence behavior of e_1 , e_2 , e_3 and e_4 in error system (3) under ten different random initial conditions. As illustrated in Fig. 6, these errors quickly converge to zero. Additionally, to simulate real-world conditions, we introduce noise into the system, focusing primarily on Gaussian noise. The outcomes under noisy conditions are presented in Fig. 7. It is evident that, despite the presence of noise, the errors rapidly converge towards zero. This demonstrates that our method exhibits excellent robustness against Gaussian noise interference. Meanwhile, the computed metrics MAE and RMSE are summarized in Table I.

Remark 6: The model's robustness to varying initial conditions stems from its training on a sufficiently large and diverse dataset, enabling it to generalize effectively across different starting states. Crucially, performance degradation

Algorithm 1 PPO Algorithm for Error Dynamic System (3)**Initialize Parameters:**

Set initial policy parameters θ_0 to define action-selection policy π .

Initialize value-function parameters ϕ_0 for state-value estimation.

Establish the initial system state s_0 of the error dynamic system (3).

Define trajectory collection Γ to store sequences of states, actions, and rewards.

Specify learning rate η , decay rates β_1 and β_2 for Adam optimizer, along with other network weights and hyperparameters.

for $k = 0, 1, 2, \dots$ **do**

 Generate trajectory collection Γ_k by executing policy $\pi_k = \pi(\theta_k)$ in the environment.

 Select action a_k based on policy π_k .

 Compute rewards-to-go r_k .

 Calculate advantage estimates A_k using current value function V_{ϕ_k} .

 Update policy parameters θ by maximizing the PPO-Clip objective:

$$\theta_{k+1} = \arg \max_{\theta} \frac{1}{|\Gamma_k|T} \sum_{\tau \in \Gamma_k} \sum_{t=0}^T \min \left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} A^{\pi_{\theta_k}}(s_t, a_t), \right. \\ \left. \text{clip} \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_k}(a|s)}, 1 - \epsilon, 1 + \epsilon \right) A^{\pi_{\theta_k}}(s, a) \right).$$

Adjust value function parameters ϕ by minimizing mean-squared error:

$$\phi_{k+1} = \arg \min_{\phi} \frac{1}{|\Gamma_k|T} \sum_{\tau \in \Gamma_k} \sum_{t=0}^T (V_{\phi}(s_t) - r_t)^2.$$

Apply Adam optimization rules for step updates to solve the function at previous step:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\phi} L(\phi_t), \\ v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\phi} L(\phi_t))^2, \\ \hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \\ \phi_{t+1} = \phi_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

end for

due to larger initial errors is mitigated as long as the training data adequately covers a broad range of initial conditions. Since our training process incorporates randomly generated initial errors, the model does not exhibit bias toward any specific error magnitude. Thus, provided the training distribution is sufficiently wide, the model maintains stable performance even when faced with larger initial deviations.

Remark 7: The DRL algorithm's robustness to Gaussian noise can be attributed to three key factors. First, its reward function penalizes both immediate absolute errors and their cumulative effect through an exponential term, ensuring

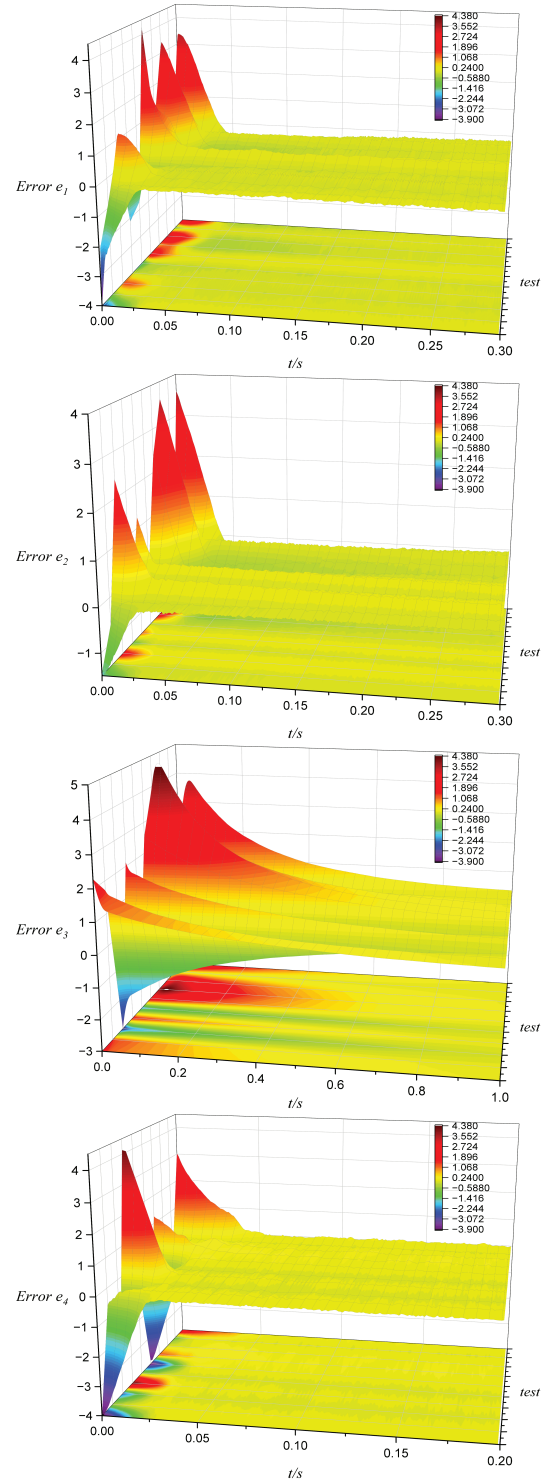


Fig. 6. The variations of errors e_1 , e_2 , e_3 and e_4 by the improved PPO algorithm.

sustained error minimization. Second, the deep neural policy inherently learns rich state representations during training, enabling generalization to noisy observations at deployment. Third, the control strategy adapts to overall system dynamics rather than reacting to instantaneous noise, which stabilizes decision-making. Notably, this robustness emerges naturally without explicit noise injection during training.

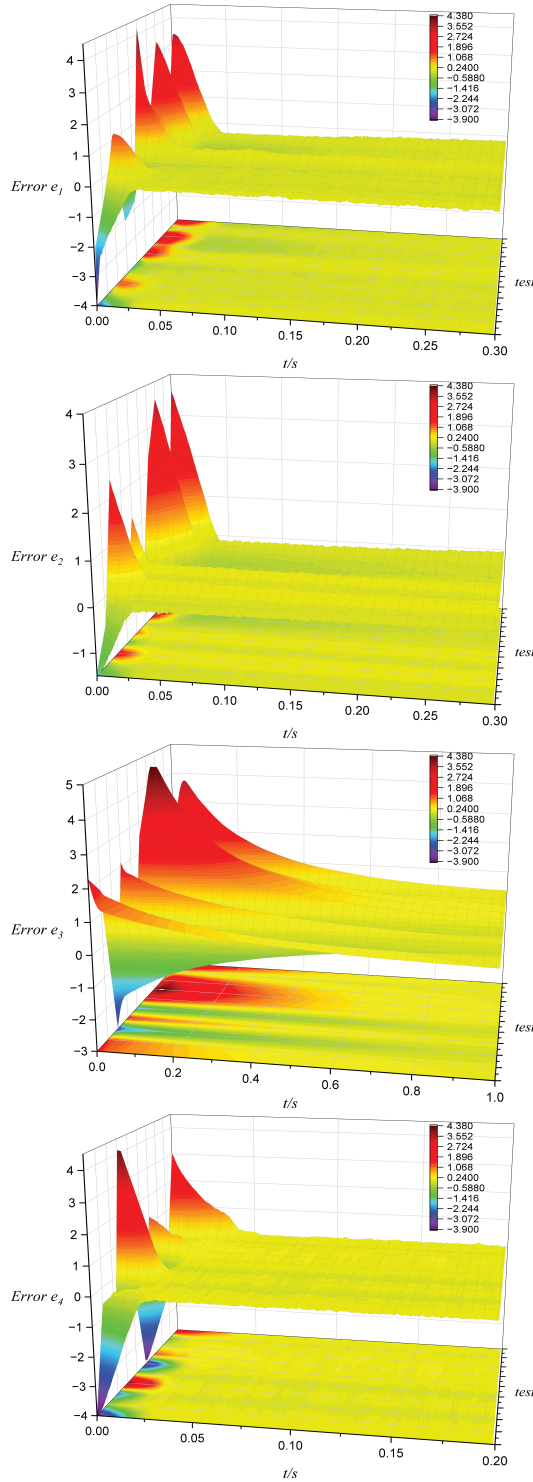


Fig. 7. The variations of errors e_1 , e_2 , e_3 and e_4 with noise by the improved PPO algorithm.

As shown in Fig. 10 from [31], which used the sliding-mode control method to solve the synchronization problem of MCSs, it is evident that at 0.1 s, 0.2 s, 0.3 s, the errors e_1 , e_2 , e_3 , and e_4 remain significant. In contrast, our method achieves much smaller errors by the same time point in table II. Therefore, it is reasonable to conclude that our method exhibits relatively ideal convergence speed under specific conditions.

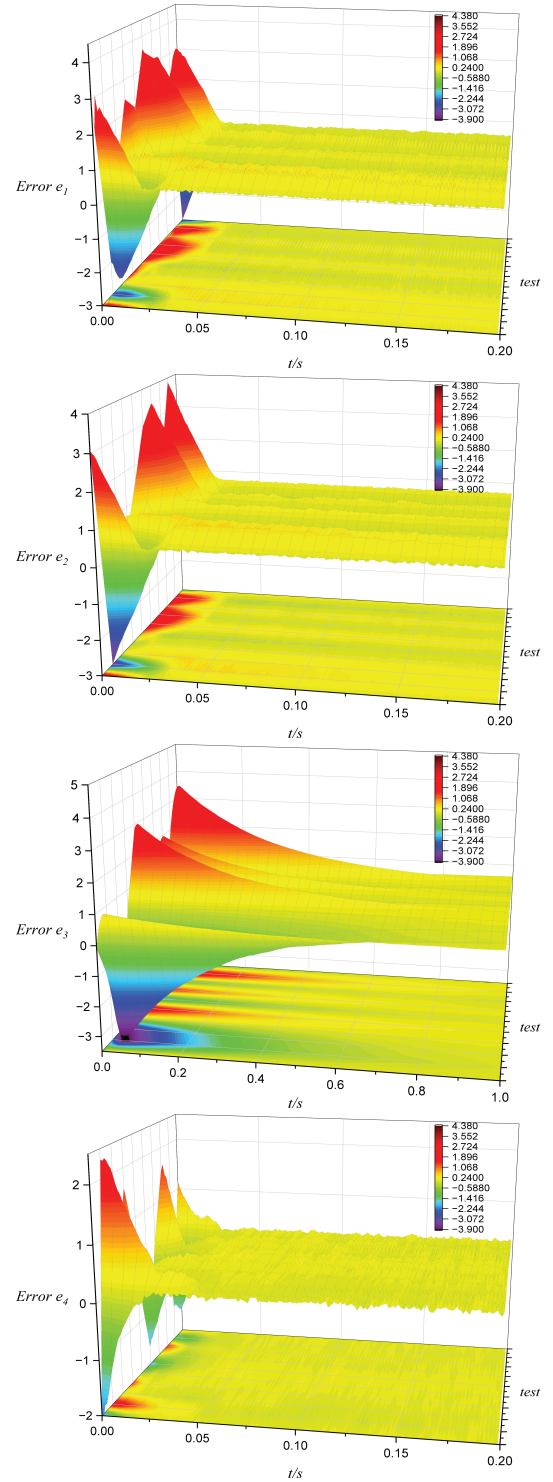


Fig. 8. The variations of errors e_1 , e_2 , e_3 and e_4 for the traditional reward (14).

In traditional DRL, one of the commonly used rewards is selected as follows:

$$r_t = -(|e_1| + |e_2| + |e_3| + |e_4|), \quad (14)$$

which is the negative Manhattan distance. Our study also experiment with this reward function for comparison. The results are illustrated in Figs. 8 and 9. It is evident that when the environment includes Gaussian noise, it is difficult for the

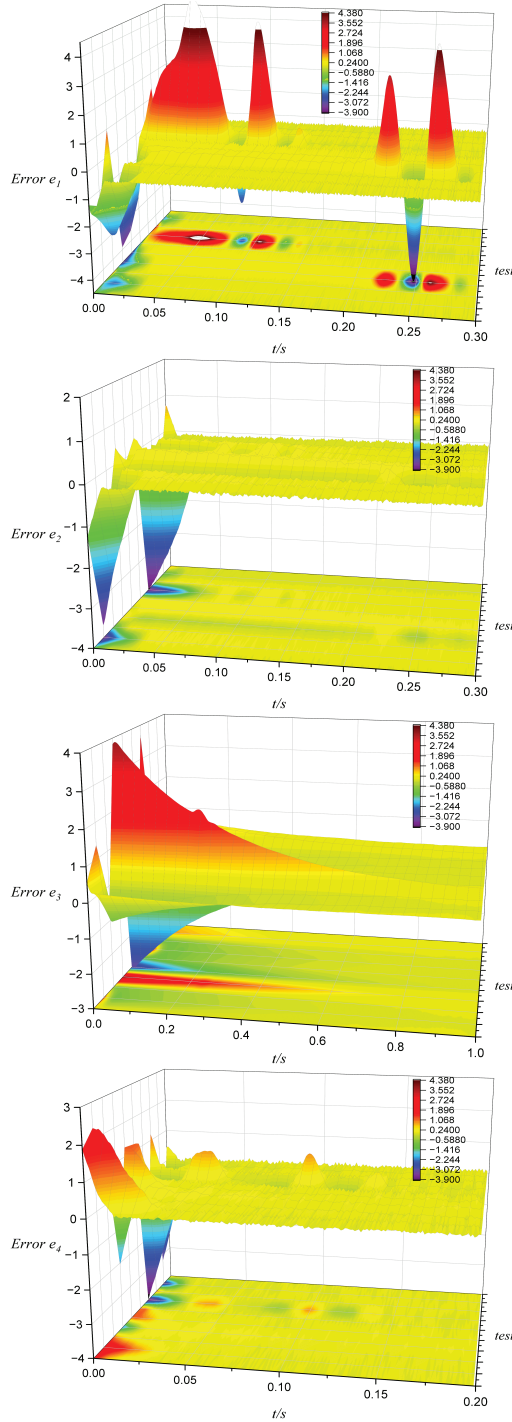


Fig. 9. The variations of errors e_1 , e_2 , e_3 and e_4 with noise for the traditional reward (14).

errors to rapidly converge towards zero. The specific MAE and RMSE results are shown in Table III.

Furthermore, different values of exponential parameter $\alpha = 1/10, 1/7, 1/5, 1/4, 1/3, 1/2$ are tested, and the results of MAE and RMSE are summarized in Fig. 11, which shows that $\alpha = 1/2$ is optimal.

IV. APPLICATION TO IMAGE ENCRYPTION

With the advancement of chaos theory, the application of chaotic synchronization in communication security has

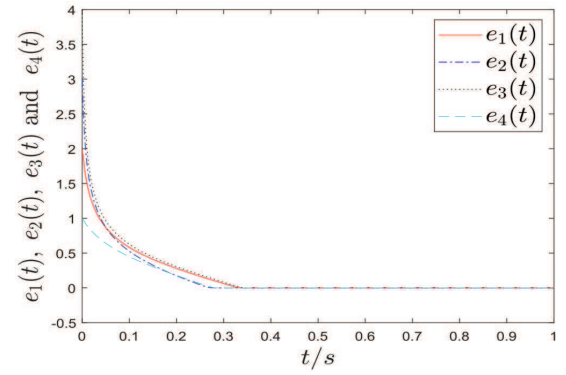


Fig. 10. The variations of errors e_1 , e_2 , e_3 and e_4 in [31].

TABLE II
THE VARIATIONS OF ERRORS e_1 , e_2 , e_3 AND e_4 AT DIFFERENT TIMES IN THIS STUDY

time	e_1	e_2	e_3	e_4
0.1 s	-0.0102	0.0537	-0.4917	-0.0185
0.2 s	0.0162	0.0152	-0.0450	0.0176
0.3 s	-0.0067	0.0287	0.0087	0.0037

TABLE III
MAE AND RMSE WITH THE TRADITIONAL MANHATTAN DISTANCE REWARD (14)

	e_1	e_2	e_3	e_4	Average
MAE	0.0355407	0.0217980	0.0165316	0.1515569	0.0886557
RMSE	0.0261422	0.0170181	0.0130122	0.4593903	0.1288907
MAE-noise	0.0568757	0.0230645	0.0162195	0.1562353	0.0952133
RMSE-noise	0.0302288	0.0176064	0.0123420	0.4881896	0.1370917

become a hot and important topic. Among the numerous encryption algorithms that have been proposed, one well-known approach is deoxyribonucleic acid (DNA) encryption, in which information is first converted into DNA sequences composed of base pairs (A, T, C, G), and each binary digit or group of digits can be mapped to specific combinations of these bases according to predefined coding rules. Once encoded, various biochemical techniques are applied to further process the DNA sequence for encryption, such as adding additional DNA fragments as keys. Only recipients with the correct key can decode the original message by identifying and removing these key sequences and then converting the DNA sequence back to its original digital form using the same coding rules. The advantages of DNA encryption include its ability to store vast amounts of information in an extremely small space, good stability allowing long-term preservation of data, and high security due to the complexity of biochemical reactions and specialized knowledge required. Table IV shows the DNA coding rules that we used.

The flow chart of our method in image encryption is shown in Fig. 12. The image encryption process begins with the original image, which is separated into its R, G, and B channels. The process then involves generating sequences using a

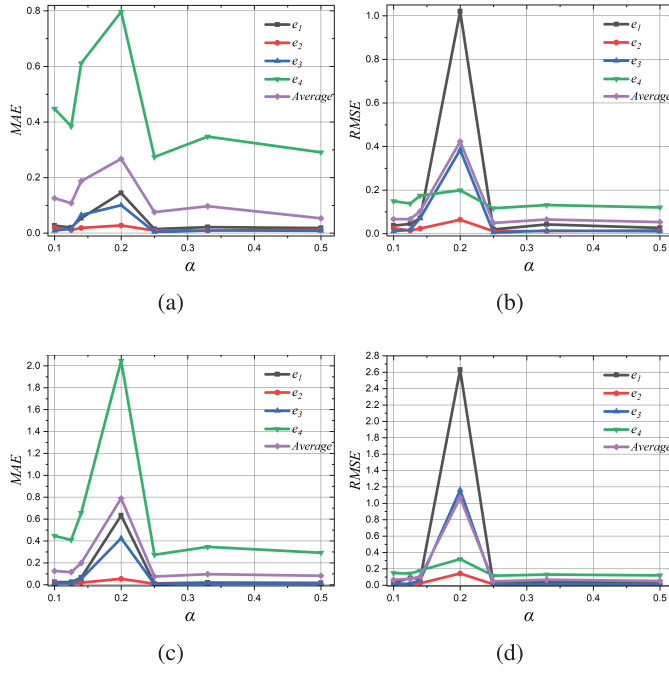


Fig. 11. The MAE and RMSE variations in the parameter α with or without noise: (a) MAE without noise, (b) RMSE without noise, (c) MAE with noise, and (d) RMSE without noise.

TABLE IV
THE CODING RULES OF DNA

Mode	1	2	3	4	5	6	7	8
00	A	A	T	T	C	G	C	G
01	G	C	G	C	A	A	T	T
10	C	G	C	G	T	T	A	A
11	T	T	A	A	G	C	G	C

logistic map and a drive circuit referring to the system (2). These sequences are further processed through exclusive OR (XOR) operations and modulo arithmetic. Subsequently, the sequences are filtered using a Kalman filter and then encoded and decoded using a DNA algorithm. The response circuit is the system (4). Finally, the encrypted image is generated. The specific steps of encryption are as follows:

- 1) The colored digital image is segmented into three separate pixel matrices, each of size $M \times N$ (where the $\times$ denotes multiplication), corresponding to the R, G, and B color channels. These matrices are denoted as I_1 , I_2 , and I_3 . Each element within these matrices has a value range from 0 to 255.
- 2) A sub-block size of $p \times p$ is defined. Consequently, each of the image pixel matrices is divided into $\frac{M \times N}{p^2}$ segments.
- 3) The Logistic map is characterized by the equation:

$$k_n = \eta k_{n-1}(1 - k_{n-1}), \quad (15)$$

where η represents the bifurcation parameter. η is set to 3.9999 as one of the encryption keys. Another key, k_0 ,

is calculated as follows:

$$k_0 = \frac{\text{Sum}(I_1) + \text{Sum}(I_2)}{255 \times M \times N \times 2}, \quad (16)$$

where $\text{Sum}(I_1)$ and $\text{Sum}(I_2)$ represent the sums of all elements in matrices I_1 and I_2 , respectively. Through the utilization of the specified parameters and initial conditions, a chaotic sequence K_1 of length $M \times N$ is generated, which is non-periodic and non-converging.

- 4) To adjust values of K_1 to the range 0-255, the following transformation is applied:

$$k'_j = \text{mod}[\text{round}(k_j), 256]. \quad (17)$$

This transformed sequence is then reshaped into a matrix Q of size $M \times N$.

- 5) An XOR operation is performed between the corresponding blocks of I_1 , I_2 , I_3 , and Q , resulting in modified matrices I'_1 , I'_2 , and I'_3 .
- 6) Solve the system (3) to generate four chaotic sequences X_1 , X_2 , X_3 , and X_4 , each of length $\frac{M \times N}{p^2}$.
- 7) One-dimensional Kalman filtering is applied each of the sequences X_1, X_2, X_3, X_4 . The implementation employs a custom one-dimensional Kalman filter that performs sequential state estimation using the standard prediction-update mechanism. Each sequence is processed independently, with the filter initialized based on the first value of the sequence. The filtering process iteratively applies the Kalman update equations to effectively reduce noise or random perturbations present in the data.
- 8) The DNA coding method for sub-blocks at identical positions in I'_1 , I'_2 , and I'_3 is determined by X_1 , X_2 . The values of X_1 , X_2 are converted to integers ranging from 1 to 8 by using

$$\begin{cases} o'_m = \text{mod}(\text{round}(o_m), 8) + 1, \\ p'_m = \text{mod}(\text{round}(p_m), 8) + 1, \end{cases} \quad (18)$$

where o_m and p_m ($m = 0, \dots, \frac{M \times N}{p^2} - 1$) denote the elements of the chaotic sequences X_1 and X_2 , respectively. The sequences I'_1 , I'_2 , and I'_3 are first encoded using DNA coding rules according to o'_m . Following this, DNA decoding is carried out based on the values of p'_m .

- 9) Repeat steps (7) and (8), substituting X_1 and X_2 with X_3 and X_4 , respectively.
- 10) Finally, the two-dimensional matrices obtained after the second substitution are combined into a three-dimensional matrix to form the encrypted image.

Remark 8: The steps for decryption are the reverse of the encryption process. In our approach, Kalman filtering plays a crucial role. Kalman filtering enhances both image encryption and chaotic sequence generation. In image encryption, it improves data integrity and mitigates noise effects by predicting and correcting errors during transmission. Kalman filtering optimizes sequence quality through noise reduction and synchronization enhancement, resulting in more unpredictable and reliable sequences. Without the Kalman filtering step, however, the image cannot be fully restored, as this

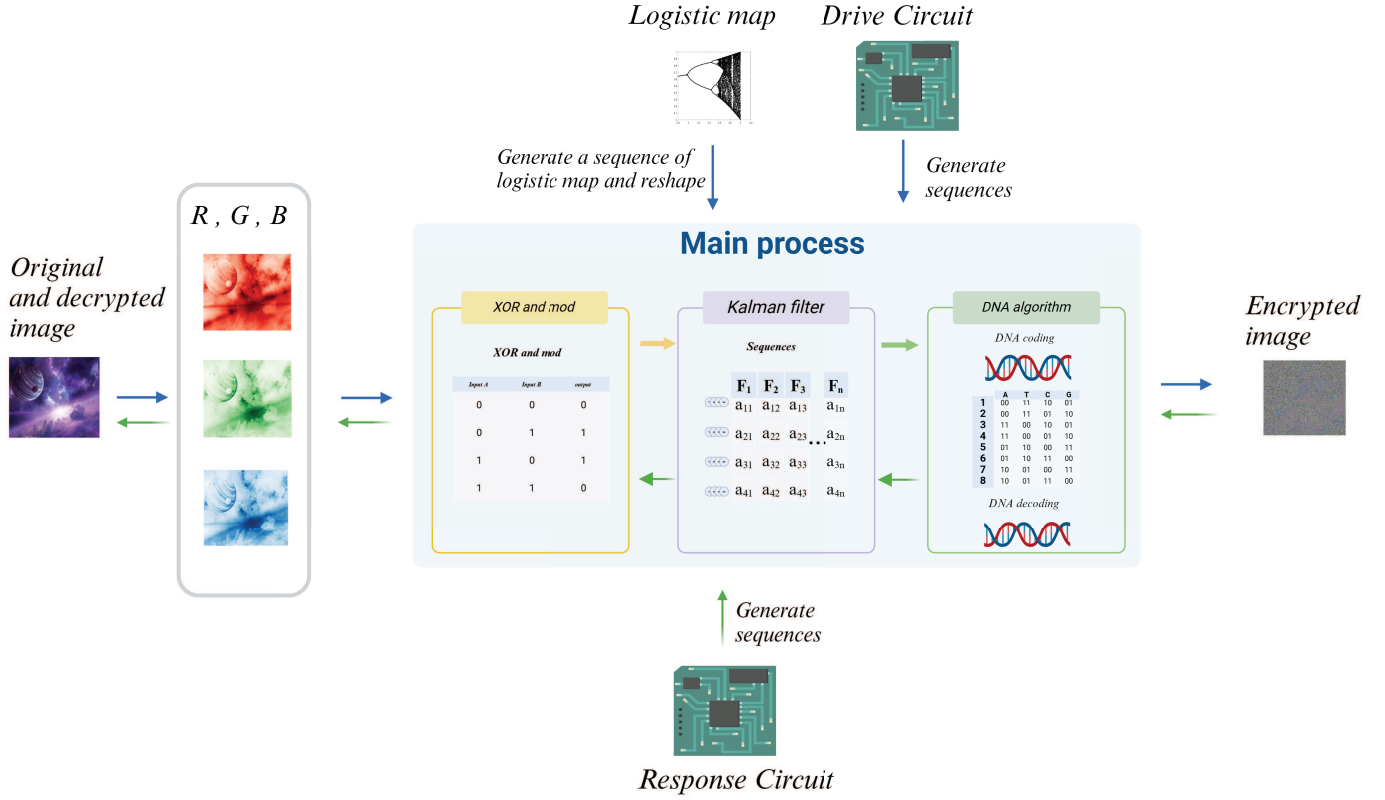


Fig. 12. The flow chart of our method in image encryption.

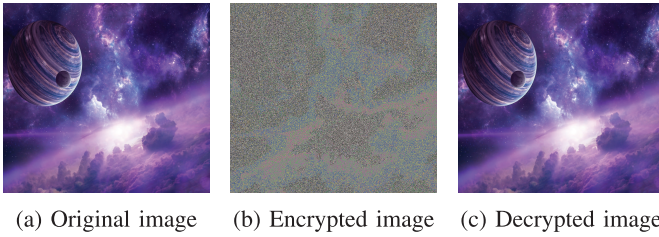


Fig. 13. Original, encrypted, and decrypted images.

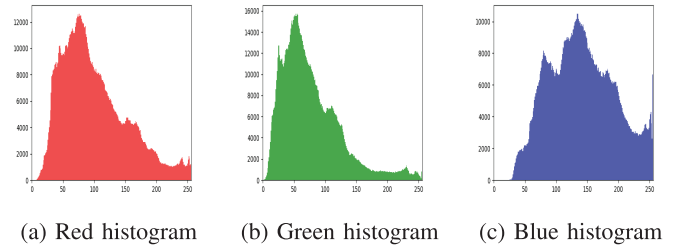


Fig. 14. Histograms of original image.

process is essential for error correction and noise reduction, ensuring the integrity of the decrypted image.

To illustrate the effectiveness of our encryption method, we apply it to a color image. As illustrated in Fig. 13(a) and Fig. 13(b), there is no significant visual similarity between the original and encrypted images, confirming the encryption's effectiveness. Fig. 13(c) demonstrates successful recovery of the original image from its encrypted form. The histograms of the encrypted image in the R, G, and B channels, shown in Fig. 14 and Fig. 15, exhibit closely resemble uniform distributions, further indicating the robustness of the encryption.

A lower correlation between the gray values of adjacent regions indicates a higher resistance of the image to various attacks. Therefore, in this section, we analyze the pixel correlation along horizontal, vertical, and diagonal axes to evaluate the encryption performance. At the same time, our comparison of previous image encryption methods primarily centers on their correlation properties. The formula for calculating the

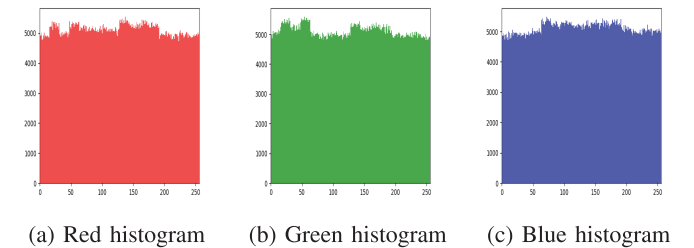


Fig. 15. Histograms of encrypted image.

correlation coefficient is given by

$$\varphi_{\delta\sigma} = \frac{\text{Cov}(\delta, \sigma)}{\sqrt{D(\delta)D(\sigma)}}, \quad (19)$$

where δ and σ represent the gray values of two adjacent pixels in the image. Here, $\text{Cov}(\delta, \sigma)$ denotes the covariance

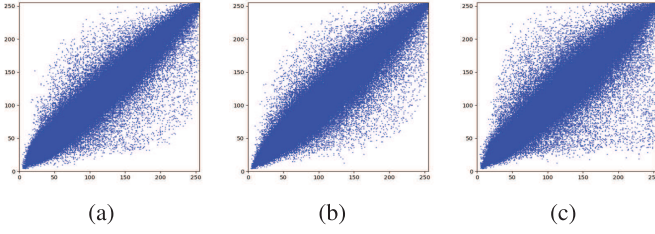


Fig. 16. Correlations of adjacent pixels in B channel of original image. (a) Horizontal direction. (b) Vertical direction. (c) Diagonal direction.

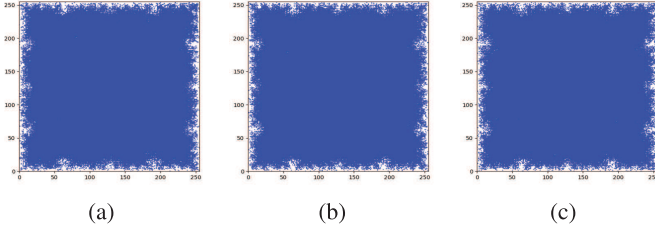


Fig. 17. Correlations of adjacent pixels in B channel of encrypted image. (a) Horizontal direction. (b) Vertical direction. (c) Diagonal direction.

TABLE V

CORRELATION COEFFICIENTS OF THE ORIGINAL AND ENCRYPTED IMAGES

Image	Horizontal	Vertical	Diagonal
Original image	0.9890	0.9878	0.9836
Encrypted image of this study	0.0056	0.0017	0.0024
Encrypted image of Ref. [31]	-0.0088	0.0047	0.0053
Encrypted Image of Ref. [41]	-0.0042	-0.0028	0.0027
Encrypted Image of Ref. [42]	-0.0015	0.0023	0.0021
Encrypted Image of Ref. [43]	-0.0123	-0.0068	-0.0368

between these pixels, while the mean $E(\delta)$ and variance $D(\delta)$ are defined as follows:

$$E(\delta) = \frac{1}{n} \sum_{i=1}^n \delta_i, \quad (20)$$

$$D(\delta) = \frac{1}{n} \sum_{i=1}^n (\delta_i - E(\delta))^2. \quad (21)$$

Figure 16 presents the distribution of adjacent pixel values in channel B of Fig. 13(a). It is evident that this distribution closely resembles a positively sloped curve, indicating a strong direct correlation among adjacent pixels. In contrast, Fig. 17 illustrates the distribution of adjacent pixel values for Fig. 13(b), showing how the encryption process disrupts such correlations.

In addition, Table V summarizes the correlation coefficients between adjacent pixels of the original and encrypted images obtained using our method, further demonstrating the effectiveness of the encryption.

To prevent attacks, an effective encryption algorithm should have a sufficiently large key space, generally assumed to be at least 2^{100} . The size of the key space can be evaluated based on the sensitivity of each individual key. Since our parameters are closely related to those in [31], our key space is also on the order of 10^{95} . There is no doubt that this design provides

an extremely large key space, making it highly resistant to exhaustive key search attacks aimed at cracking the image.

V. CONCLUSION

This paper puts forward an AI-based control strategy aimed at synchronizing two MCSs without the requirement of prior knowledge about their dynamic behaviors. Different from traditional linear and nonlinear control techniques that rely on precise mathematical models, the proposed approach leverages DRL. By integrating only the essential control signals, the improved PPO algorithm significantly reduces the complexity of system synchronization.

Specifically, the method is based on an enhanced DRL approach with an improved PPO algorithm. It simplifies the control architecture by reducing the number of control forces compared to the conventional item-by-item control method. Additionally, the classical PPO algorithm is enhanced through refining the reward function, which not only enables MCSs to synchronize with a faster convergence rate but also endows them with robustness against Gaussian noise interference.

The effectiveness of this method has been verified through its successful application in image encryption. In this application, the synchronized MCSs show robust performance and enhanced security features. Given that real-world situations often involve unpredictable perturbations, future research can focus on training these chaotic systems in noisy environments to further enhance their resilience. Another promising research direction is to explore alternative DRL methods to achieve better convergence properties in chaotic system synchronization. These advancements have the potential to be extended to a broader range of applications, such as secure communications and data protection.

REFERENCES

- [1] E. N. Lorenz, "Deterministic nonperiodic flow," *J. Atmos. Sci.*, vol. 20, no. 2, pp. 130–141, Mar. 1963.
- [2] J. P. Crutchfield, "Between order and chaos," *Nature Phys.*, vol. 8, no. 1, pp. 17–24, Jan. 2011.
- [3] J. M. Ottino, F. J. Muzzio, M. Tjahjadi, J. G. Franjione, S. C. Jana, and H. A. Kusch, "Chaos, symmetry, and self-similarity: Exploiting order and disorder in mixing processes," *Science*, vol. 257, no. 5071, pp. 754–760, Aug. 1992.
- [4] E. Benincà et al., "Chaos in a long-term experiment with a plankton community," *Nature*, vol. 451, no. 7180, pp. 822–825, Feb. 2008.
- [5] D. A. Hsieh, "Chaos and nonlinear dynamics: Application to financial markets," *J. Finance*, vol. 46, no. 5, pp. 1839–1877, Dec. 1991.
- [6] S. H. Strogatz, *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering*. New York, NY, USA: CRC Press, 2024.
- [7] T. Umar, M. Nadeem, and F. Anwer, "Chaos based image encryption scheme to secure sensitive multimedia content in cloud storage," *Exp. Syst. Appl.*, vol. 257, Aug. 2024, Art. no. 125050.
- [8] X. Liu, C. Li, S. S. Ge, and D. Li, "Time-synchronized control of chaotic systems in secure communication," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 9, pp. 3748–3761, Sep. 2022.
- [9] Z. Zhong, W. Lin, and B.-W. Qin, "Modulating biological rhythms: A noncomputational strategy harnessing nonlinearity and decoupling frequency and amplitude," *Phys. Rev. Lett.*, vol. 131, no. 13, Sep. 2023, Art. no. 138401.
- [10] L. Chua, "Memristor—The missing circuit element," *IEEE Trans. Circuit Theory*, vol. CT-18, no. 5, pp. 507–519, Sep. 1971.
- [11] D. B. Strukov, G. S. Snider, D. R. Stewart, and R. S. Williams, "The missing memristor found," *Nature*, vol. 453, no. 7191, pp. 80–83, May 2008.

- [12] L. V. Gambuzza, A. Buscarino, L. Fortuna, and M. Frasca, "Memristor-based adaptive coupling for consensus and synchronization," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 62, no. 4, pp. 1175–1184, Apr. 2015.
- [13] M. Di Marco, M. Forti, and L. Pancioni, "New conditions for global asymptotic stability of memristor neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 5, pp. 1822–1834, May 2018.
- [14] B. Muthuswamy and P. Kokate, "Memristor-based chaotic circuits," *IETE Tech. Rev.*, vol. 26, no. 6, pp. 417–429, Dec. 2009.
- [15] B. Bao, Z. Ma, J. Xu, Z. Liu, and Q. Xu, "A simple memristor chaotic circuit with complex dynamics," *Int. J. Bifurcation Chaos*, vol. 21, no. 9, pp. 2629–2645, Sep. 2011.
- [16] H. Li, L. Wang, and S. Duan, "A memristor-based scroll chaotic system—Design, analysis and circuit implementation," *Int. J. Bifurcation Chaos*, vol. 24, no. 7, Jul. 2014, Art. no. 1450099.
- [17] S. Yang, C. Li, and T. Huang, "Impulsive control and synchronization of memristor-based chaotic circuits," *Int. J. Bifurcation Chaos*, vol. 24, no. 12, Dec. 2014, Art. no. 1450162.
- [18] L. Wang, Z. Zeng, M.-F. Ge, and J. Hu, "Global stabilization analysis of inertial memristive recurrent neural networks with discrete and distributed delays," *Neural Netw.*, vol. 105, pp. 65–74, Sep. 2018.
- [19] S. Wen, Z. Zeng, and T. Huang, "Adaptive synchronization of memristor-based Chua's circuits," *Phys. Lett. A*, vol. 376, no. 44, pp. 2775–2780, Sep. 2012.
- [20] S. Wen, Z. Zeng, T. Huang, and Y. Chen, "Fuzzy modeling and synchronization of different memristor-based chaotic circuits," *Phys. Lett. A*, vol. 377, nos. 34–36, pp. 2016–2021, Nov. 2013.
- [21] W. Wang et al., "Fixed-time synchronization of fractional order memristive MAM neural networks by sliding mode control," *Neurocomputing*, vol. 401, pp. 364–376, Aug. 2020.
- [22] Y. Xin and Z. Cheng, "Adaptive synchronization for delayed chaotic memristor-based neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 2, pp. 601–610, Feb. 2023.
- [23] L. Wang, Q. Wang, and G. Zhang, "Adaptive intermittent stabilization of memristive chaotic system via T-S fuzzy model," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 3, pp. 1351–1355, Mar. 2024.
- [24] C. Li, F. Min, Q. Jin, and H. Ma, "Extreme multistability analysis of memristor-based chaotic system and its application in image decryption," *AIP Adv.*, vol. 7, no. 12, Dec. 2017, Art. no. 125204.
- [25] H. Liu, H. Wan, C. K. Tse, and J. Lü, "An encryption scheme based on synchronization of two-layered complex dynamical networks," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 63, no. 11, pp. 2010–2021, Nov. 2016.
- [26] L. Wang, T. Dong, and M.-F. Ge, "Finite-time synchronization of memristor chaotic systems and its application in image encryption," *Appl. Math. Comput.*, vol. 347, pp. 293–305, Apr. 2019.
- [27] M. E. Sahin, Z. G. Cam Taskiran, H. Guler, and S. E. Hamamci, "Application and modeling of a novel 4D memristive chaotic system for communication systems," *Circuits, Syst., Signal Process.*, vol. 39, no. 7, pp. 3320–3349, Jan. 2020.
- [28] M. Yuan, W. Wang, Z. Wang, X. Luo, and J. Kurths, "Exponential synchronization of delayed memristor-based uncertain complex-valued neural networks for image protection," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 151–165, Jan. 2021.
- [29] J. Luo, S. Qu, Y. Chen, and Z. Xiong, "Synchronization of memristor-based chaotic systems by a simplified control and its application to image en-/decryption using DNA encoding," *Chin. J. Phys.*, vol. 62, pp. 374–387, Dec. 2019.
- [30] G. Peng and F. Min, "Multistability analysis, circuit implementations and application in image encryption of a novel memristive chaotic circuit," *Nonlinear Dyn.*, vol. 90, no. 3, pp. 1607–1625, Nov. 2017.
- [31] L. Wang, S. Jiang, M.-F. Ge, C. Hu, and J. Hu, "Finite-/fixed-time synchronization of memristor chaotic systems and image encryption application," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 12, pp. 4957–4969, Dec. 2021.
- [32] S. Gao et al., "A 3D memristive cubic map with dual discrete memristors: Design, implementation, and application in image encryption," *IEEE Trans. Circuits Syst. Video Tech.*, early access, Feb. 26, 2025, doi: 10.1109/TCSVT.2025.3545868.
- [33] M. A. Bucci, O. Semeraro, A. Allauzen, G. Wisniewski, L. Cordier, and L. Mathelin, "Control of chaotic systems by deep reinforcement learning," *Proc. Roy. Soc. A*, vol. 475, no. 2231, Jun. 2019, Art. no. 20190351.
- [34] H. Cheng, H. Li, Q. Dai, and J. Yang, "A deep reinforcement learning method to control chaos synchronization between two identical chaotic systems," *Chaos, Solitons Fractals*, vol. 174, Sep. 2023, Art. no. 113809.
- [35] H.-T. Yau, P.-H. Kuo, P.-C. Luan, and Y.-R. Tseng, "Proximal policy optimization-based controller for chaotic systems," *Int. J. Robust Nonlinear Control*, vol. 34, no. 1, pp. 586–601, Jan. 2024.
- [36] S. Vashishtha and S. Verma, "Restoring chaos using deep reinforcement learning," *Chaos, Interdiscipl. J. Nonlinear Sci.*, vol. 30, no. 3, Mar. 2020, Art. no. 031102.
- [37] L. Xu, R. Ma, J. Wu, and P. Rao, "Proximal policy optimization approach to stabilize the chaotic food web system," *Chaos Solitons Fractals*, vol. 192, Mar. 2025, Art. no. 116033.
- [38] M. Itoh and L. O. Chua, "Memristor oscillators," *Int. J. Bifurcation Chaos*, vol. 18, no. 11, pp. 3183–3206, Nov. 2008.
- [39] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [40] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," 2015, *arXiv:1506.02438*.
- [41] W. Hao, T. Zhang, X. Chen, and X. Zhou, "A hybrid NEQR image encryption cryptosystem using two-dimensional quantum walks and quantum coding," *Signal Process.*, vol. 205, Apr. 2023, Art. no. 108890.
- [42] L. Li, "A novel chaotic map application in image encryption algorithm," *Expert Syst. Appl.*, vol. 252, Oct. 2024, Art. no. 124316.
- [43] V. Verma and S. Kumar, "Quantum image encryption algorithm based on 3D-BNM chaotic map," *Nonlinear Dyn.*, vol. 113, no. 4, pp. 3829–3855, Feb. 2025.



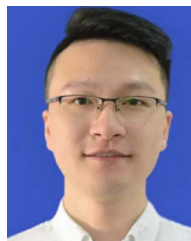
Shitao Jin received the B.S. degree from the School of Computer Science, Yangtze University, Jingzhou, China, in 2023. He is currently pursuing the M.S. degree in software engineering with the School of Artificial Intelligence and Computer Science, Jiangnan University.

His current research interests include stability and synchronization of chaotic systems and complex networks with applications.



Jie Chen received the M.S. degree from the School of Automation, Central South University, Changsha, China, in 2018, and the Ph.D. degree from School of Information Science and Engineering, Fudan University, Shanghai, China, in 2023.

He is currently a Lecturer at the National University of Defense Technology. His current research interests include multiagent task assignment, evolutionary game theory, and game-theoretical optimization of complex networks.



Jie Wu received the B.S. degree in applied mathematics from Hainan Normal University, Haikou, China, in 2013, the M.S. degree in mathematics from China University of Mining and Technology, Xuzhou, China, in 2015, and the Ph.D. degree in circuits and system from Fudan University, Shanghai, China, in 2020.

Since June 2020, he has been joining the School of Artificial Intelligence and Computer Science, Jiangnan University, Wuxi, China, where he is currently an Associate Professor. His current research

interests include stability and synchronization of chaotic systems and complex networks with applications.



Xiaofeng Wang was born in Wuxi, Jiangsu, China, in 1978. He received the B.S. degree in communication engineering, the M.S. degree in communication and information system, and the Ph.D. degree in computer system structure from Harbin Institute of Technology, Harbin, China, in 2001, 2003, and 2007, respectively.

He is currently a Research Fellow at the Peng Cheng Laboratory, Shenzhen, China. His research interests include network science, artificial intelligence, and network security with applications.



Junjie Fu (Senior Member, IEEE) received the B.S. and Ph.D. degrees from Peking University, Beijing, China, in 2011 and 2017, respectively.

He is currently a Professor with the School of Mathematics, Southeast University, Nanjing, China. His research interests include coordination of multi-agent systems, distributed collision avoidance, and multi-agent reinforcement learning. He served as a program committee member for several international conferences. He was a recipient of the APNNS Young Researcher Award and has received several

best conference paper awards, including the 2022 IEEE ICUS Best Paper Award and the 2020 IEEE ICCSS Zadeh Best Conference Paper Award. He is an Associate Editor of *IEEE Systems, Man, and Cybernetics Magazine* and a Youth Editor of *The Innovation*.



Xiaoli Luan (Member, IEEE) received the B.Sc. degree in industrial automation and the M.Sc. and Ph.D. degrees in control theory and control engineering from Jiangnan University, Wuxi, China, in 2002, 2006, and 2010, respectively.

In 2016, she was a Visiting Professor with the University of Alberta, Edmonton, AB, Canada. She is currently a Professor with the Institute of Automation and the Dean of the School of Internet of Things Engineering, Jiangnan University. Her research interests include robust control and optimization of

complex industrial process. She has authored or co-authored more than 100 papers in professional journals, conference proceedings, and technical reports in the above areas.



Guanghui Wen (Senior Member, IEEE) received the Ph.D. degree in mechanical systems and control from Peking University, Beijing, China, in 2012.

He is currently the Endowed Chair Professor and the Vice Dean of the School of Automation, Southeast University, Nanjing, China. His current research interests include autonomous intelligent systems, complex networked systems, distributed control and optimization, resilient control, and distributed reinforcement learning.

Prof. Wen is a fellow of IET. He was a recipient of the National Science Fund for Distinguished Young Scholars, China Youth Science and Technology Award, Australian Research Council Discovery Early Career Researcher Award, and Asia-Pacific Neural Network Society Young Researcher Award. He serves as a Technical Editor for IEEE/ASME TRANSACTIONS ON MECHATRONICS and an Associate Editor for IEEE TRANSACTIONS ON CONTROL OF NETWORK SYSTEMS, IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, IEEE TRANSACTIONS ON INTELLIGENT VEHICLES, IEEE TRANSACTIONS ON FUZZY SYSTEMS, IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS: SYSTEMS, IEEE OPEN JOURNAL OF THE INDUSTRIAL ELECTRONICS SOCIETY, and *Asian Journal of Control*. He has been named a Highly Cited Researcher by Clarivate Analytics since 2018.